SPRINGLOG-IA

# Solución con IA generativa

*Prompts especializados, ejemplos, capacidades y limitaciones*

| | | | |
|---|---|---|---|
| **ENTREGABLE** | 02 | **ESTUDIANTE** | David Felipe Rios Echeverri |
| **ÁREA** | Desarrollo Backend / Ingeniería de Software | **FECHA** | Octubre de 2026 |

**Propósito**

Explorar cómo la IA generativa puede estructurar logs, sugerir hipótesis de diagnóstico, ayudar a sanitizar datos y resumir patrones. Sus salidas se consideran asistencia para un ingeniero, no evidencia por sí solas de la causa raíz.

## 1. Contexto de la exploración

La exploración se realiza sobre un conjunto sintético de 500 registros de payment-service. Los ejemplos de entrada y salida de este informe son ejemplos reproducibles de diseño de interacción, no una prueba comparativa de modelos generativos ni un resultado validado sobre telemetría de producción.

## 2. Prompts especializados

### Prompt 1 — Extracción estructurada y clasificación

**Prompt**

Actúa como ingeniero SRE sénior experto en Java y Spring Boot 3. Analiza el evento recibido y devuelve únicamente JSON válido con timestamp, app_name, instance_id, thread, logger_class, exception_type, severity_original, severity_sugerida, resumen_tecnico y confianza. Conserva la severidad original. Usa null o NO_APLICA si falta un campo. No inventes información. Separa los hechos observados de cualquier inferencia. Log: [PEGAR LOG SANITIZADO].

### Prompt 2 — Diagnóstico de causa raíz

**Prompt**

Actúa como arquitecto backend experto en Spring Data JPA, Hibernate y HikariCP. A partir de la traza, separa hechos, hipótesis e información faltante. Entrega causa raíz probable, evidencia textual, impacto potencial, pruebas de diagnóstico priorizadas y métricas para verificar la hipótesis. No propongas cambios de configuración sin evidencia suficiente. Traza: [PEGAR STACK TRACE SANITIZADO].

### Prompt 3 — Sanitización de datos sensibles

**Prompt**

Actúa como responsable de seguridad. Detecta y enmascara JWT/Bearer, tokens, contraseñas, claves API, correos, identificadores personales y números de tarjeta. Sustituye los valores por marcadores [REDACTED_*], conserva el contexto técnico no sensible y devuelve la línea sanitizada y la lista de tipos de dato ocultados. No reproduzcas los valores originales. Log: [PEGAR LOG].

### Prompt 4 — Resumen operativo

**Prompt**

Actúa como líder técnico de payment-service. Resume los registros suministrados por severidad e identifica tipos de excepción y señales operativas recurrentes. Diferencia conteos del dataset, hipótesis de impacto y recomendaciones. No sumes categorías solapadas como si fueran incidentes únicos. Declara que la muestra es sintética. Datos: [PEGAR TABLA O LOGS].

## 3. Ejemplos de entrada y salida

### 3.1. Caso de pool de conexiones

Entrada sintética: “TransactionRepository: Connection is not available; HikariPool timeout after 30000 ms; activeConnections=20; maxPoolSize=20”.

Salida ilustrativa: severidad sugerida CRITICAL si coincide con la etiqueta y el contexto de operación; síntoma observado: no se consiguió una conexión dentro del tiempo límite; hipótesis por verificar: saturación del pool, consultas lentas o transacciones largas; acción inicial: revisar métricas active/idle/pending/timeouts y límites de la base de datos antes de cambiar parámetros.

### 3.2. Caso de sanitización

Entrada ficticia: “Payment failed userEmail=ana@example.test token=abc123secret customerId=9281”.

Salida esperada: “Payment failed userEmail=[REDACTED_EMAIL] token=[REDACTED] customerId=[REDACTED_ID]”.

### 3.3. Resumen de la muestra

| Severidad | Registros | Porcentaje |
|---|---|---|
| INFO | 300 | 60 % |
| WARN | 100 | 20 % |
| ERROR | 75 | 15 % |
| CRITICAL | 25 | 5 % |

La proporción de severidades fue definida al generar el dataset; no es una estimación de la frecuencia de eventos de producción.

## 4. Capacidades observadas y límites

| Capacidad | Utilidad | Limitación |
|---|---|---|
| Estructuración | Normaliza campos y genera JSON tabular. | Puede inventar o inferir campos si el prompt no restringe la salida. |

| Capacidad | Utilidad | Limitación |
|---|---|---|
| Diagnóstico asistido | Propone hipótesis y pasos de verificación. | Un log aislado no demuestra causa raíz. |
| Resumen | Sintetiza frecuencias y patrones. | Categorías por palabras clave pueden solaparse. |
| Sanitización | Ayuda a revisar la política de enmascaramiento. | Un LLM no debe ser el único control; se requiere filtro determinista y pruebas. |
| Escalabilidad | Puede asistir análisis complejos. | Costo, latencia, límites de API y privacidad impiden asumir que procesará cada log en tiempo real. |

## 5. Próximos pasos

- Aplicar sanitización determinista antes de cualquier servicio externo.
- Versionar prompts y conservar ejemplos de evaluación con datos sintéticos.
- Comparar resultados generativos con el clasificador supervisado usando el mismo conjunto de casos.
- Requerir revisión humana para CRITICAL, confianza baja o recomendaciones que impliquen cambios operativos.